

Exercise 1 – Signals & Slots (GUI Event Communication)

Concept

- Signals and Slots

Problem Statement

Create a Qt Widgets application with:

- A QPushButton labeled **“Toggle Status”**
- A QLabel showing **“OFF”** initially

Requirements

1. When the button is clicked:
 - a. Toggle label text between **“ON”** and **“OFF”**
2. Use:
 - a. One signal
 - b. One slot
3. No global variables allowed.

Learning Outcome

- Event-driven programming
- Signal-slot connection mechanism

Exercise 2 – Reflection (Qt Meta-Object System)

Concept

- Reflection using Qt

Problem Statement

Create a GUI application with:

- A QPushButton labeled **“Inspect Object”**
- A QTextEdit to display results

Requirements

1. Create a Device class derived from QObject
2. Add at least **two Q_PROPERTY members**
3. On button click:
 - a. Display:
 - i. Class name
 - ii. Property names and types
 - b. Use QMetaObject and QMetaProperty

Learning Outcome

- Runtime introspection
- Role of Q_PROPERTY

Exercise 3 – Collections & JSON File Handling

Concept

- Qt Collections
- JSON read/write
- File handling

Problem Statement

Create a GUI application with:

- Input fields: **Product Name, Price**
- Buttons: **Add Product, Save, Load**

- QTableWidget to display products

Requirements

1. Store products in:
 - a. `QList<QMap<QString, QVariant>>`
2. Save product list to a **JSON file**
3. Load JSON file and repopulate the table

Learning Outcome

- Working with Qt containers
- Persisting data using JSON

Exercise 4 – Multithreading (No Synchronization)

Concept

- QThread
- Parallel execution

Problem Statement

Create a dashboard GUI with:

- Three labels: **Worker 1**, **Worker 2**, **Worker 3**
- One **Start** button

Requirements

1. Each worker runs in a separate thread
2. Every thread:
 - a. Emits a random number every second
3. Update the UI using signals (no direct UI access)

Learning Outcome

- Thread creation
- Signal-slot communication across threads

Exercise 5 – Mutex (Mutual Exclusion Only)

Concept

- QMutex
- Race condition prevention

Problem Statement

Create a GUI application with:

- One **shared counter**
- Two worker threads
- A label showing the counter value

Requirements

1. Both threads increment the same counter
2. Protect the counter using **QMutex**
3. Show:
 - a. Correct sequential increments
4. Remove the mutex and observe incorrect behavior (discussion point)

Learning Outcome

- Critical section concept
- How mutex prevents race conditions

Exercise 6 – Semaphore (Resource Limiting Only)

Concept

- QSemaphore
- Limited resource access

Problem Statement

Create a GUI application with:

- Four worker threads
- One label or log area showing **active workers**

Requirements

1. Use a QSemaphore(2):
 - a. Only **two threads** can work at the same time
2. Display:
 - a. Which threads are currently running
3. Other threads must wait until a permit is released

Learning Outcome

- Counting semaphore concept
- Difference between mutex and semaphore